

Indian Institute of Information Technology
Vadodara .

CampusEats

CS543 — Project Submission Report

Assignments 01–06

Team ID: Group No. 10

Submitted By:

- Brajakshi Mandloi — Roll No. 20252651016
- Vaishnavi Tiwari — Roll No. 20252651061

GitHub Repository:
https://github.com/brajakshi1427-alt/campuseats

Project Demonstration Video:
[Paste your Google Drive demonstration video link here]

---

1. Introduction

CampusEats is a software project developed for the CS543 coursework. The project brings together software design, database modelling, service integration, REST API development, HTTP protocol features, error handling, and testing.

The purpose of the project is to apply web services and software engineering concepts to a campus food ordering scenario. The project documentation and implementation artifacts are maintained in the CampusEats GitHub repository.

This report summarizes the project and its relationship to Assignments 01–06. It also describes the external SOAP payment-partner integration prepared for Assignment 3, along with the relevant technical artifacts and submission information.

2. Project Objectives

The main objectives of CampusEats are:

1. To organize the project using a clear software structure.
2. To design the application architecture and database schema.
3. To describe integration with an external payment service using SOAP.
4. To provide REST-based resources for application operations.
5. To apply HTTP methods, headers, caching, CORS, and rate-limiting concepts where required.
6. To implement structured error responses and input validation.
7. To document setup, testing, and project artifacts for evaluation.

3. Team Details

Team Member| Roll Number
Brajakshi Mandloi| 20252651016
Vaishnavi Tiwari| 20252651061

Team: Group No. 10

Course: CS543 — Web Services

Project: CampusEats

4. GitHub Repository and Project Structure

The source code and supporting materials are maintained in the following repository:

https://github.com/brajakshi1427-alt/campuseats

The project repository contains backend and frontend-related files, database and service-design materials, assignment-specific documentation, and testing resources.

The following table describes the main areas identified in the project materials.

Directory or artifact| Purpose
"assignment2/"| Architecture and database-design artifacts
"assignment3/"| SOAP partner integration documentation and XML/WSDL files
"assignment4/"| REST API assignment materials
"assignment5/"| HTTP protocol and API behaviour assignment materials
"assignment6/"| Error-handling and validation assignment materials
"backend/"| Backend-related code, services, database resources, and tests
"frontend/"| Frontend application resources
"docs/" and supporting files| Project documentation and diagrams
"README.md"| Repository overview and setup guidance

The final repository should be checked to ensure that the filenames and directory names in this report match the files committed to GitHub.

5. Assignment-Wise Summary

5.1 Assignment 1 — Project Foundation

The first assignment forms part of the foundation for the CampusEats project. The project repository and its documentation provide the starting point for organizing the work completed across the subsequent assignments.

The project foundation supports the overall campus food ordering scenario and provides a common context for the database design, service integration, API operations, and error-handling work.

Submission evidence: Refer to the project repository and the original Assignment 1 submission materials.

Note: The exact Assignment 1 brief and its individual implementation requirements should be checked against the submitted assignment before this section is finalized.

5.2 Assignment 2 — System Design and Database Schema

Assignment 2 covers the design-related materials for CampusEats.

The available artifacts include a design document, SQL schema, database schema diagram, and service architecture diagram. These materials describe the structure of the application and the relationship between its data and services.

The database schema provides a structured representation of the data required by the application. The service diagram illustrates how application components are organized and how they interact.

Relevant artifacts include:

- "design.pdf"
- "schema.sql"
- "services.drawio"
- "services.png"
- "schema.drawio"
- "schema.png"

These filenames should be adjusted if the committed repository uses different names.

Outcome: The design and schema artifacts document the intended application structure and provide a reference for implementation.

5.3 Assignment 3 — Integrate an External SOAP Partner

5.3.1 Context

CampusEats integrates with an external payment partner named CampusPay Payment Gateway. The single operation considered in this assignment is "chargePayment", which requests payment for a CampusEats order.

SOAP is selected for this external payment integration because the partner contract is described using a WSDL contract, SOAP headers can carry message-level authentication information, and SOAP provides structured fault handling suitable for a payment operation.

The remaining CampusEats services can continue using REST because internal application services do not necessarily require the same external SOAP contract.

5.3.2 SOAP Operation

The operation used in this integration is:

"chargePayment"

The request contains the order identifier, amount, currency, and payment token. The response returns a transaction identifier, payment status, and message.

5.3.3 HTTP Binding

The SOAP message is carried using HTTP POST.

POST /soap/payment HTTP/1.1
Host: api.campus-pay.example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://campus-pay.example.com/payment/chargePayment"

Endpoint:

https://api.campus-pay.example.com/soap/payment

The SOAPAction identifies the payment operation being requested.

5.3.4 Service Discovery

CampusEats would obtain the partner contract either from a WSDL URL supplied directly by the partner or through a service catalogue or registry lookup.

A live UDDI server is not required for this design. The following registry-style record acts as a pointer to the technical contract.

Field| Value
Business| CampusPay
Service| Payment Gateway SOAP Service
Endpoint| "https://api.campus-pay.example.com/soap/payment"
WSDL pointer| "https://api.campus-pay.example.com/wsdl/payment.wsdl"

The WSDL describes the service contract, including the operation, messages, binding, and service endpoint.

5.3.5 SOAP Request

The following XML represents a sample request. Authentication information is placed in the SOAP Header, while the payment operation and its data are placed in the SOAP Body.

File: "soap-request.xml"

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
   xmlns:pay="http://campus-pay.example.com/payment">

 <soap:Header>
  <pay:Authentication>
   <pay:merchantId>CE-MERCHANT-001</pay:merchantId>
   <pay:apiKey>DEMO-KEY-12345</pay:apiKey>
  </pay:Authentication>
 </soap:Header>

 <soap:Body>
  <pay:chargePayment>
   <pay:orderId>ORD-1001</pay:orderId>
   <pay:amount>500.00</pay:amount>
   <pay:currency>INR</pay:currency>
   <pay:cardToken>TOKEN-987654</pay:cardToken>
  </pay:chargePayment>
 </soap:Body>

</soap:Envelope>
```

The example uses a demonstration merchant identifier, API key, order identifier, and token. These values are illustrative and must not be treated as production credentials.

5.3.6 Successful SOAP Response

The following XML represents a successful payment response.

File: "soap-response.xml"

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
   xmlns:pay="http://campus-pay.example.com/payment">

 <soap:Body>
  <pay:chargePaymentResponse>
```

```
      <pay:transactionId>TXN-20260821-1001</pay:transactionId>
      <pay:status>SUCCESS</pay:status>
      <pay:message>Payment successful</pay:message>
    </pay:chargePaymentResponse>
  </soap:Body>

</soap:Envelope>
```

The response communicates that the sample transaction succeeded. The transaction identifier can be used as a reference in the payment workflow.

5.3.7 SOAP Fault Response

A payment request may fail if the payment instrument is declined. SOAP Fault provides a structured representation of this failure.

File: "soap-fault.xml"

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:pay="http://campus-pay.example.com/payment">

  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Payment could not be completed</faultstring>
      <detail>
        <pay:PaymentFault>
          <pay:code>card_declined</pay:code>
          <pay:description>The payment instrument was declined by the issuer.</pay:description>
        </pay:PaymentFault>
      </detail>
    </soap:Fault>
  </soap:Body>

</soap:Envelope>
```

The fault contains a general error message and structured details identifying the partner-specific failure.

5.3.8 Fault Mapping

If CampusPay returns the partner-specific fault "card_declined", CampusEats translates it at the integration boundary into its own contract-level error:

"PAYMENT_DECLINED"

The user-facing message is:

“Payment declined. Please try another payment method.”

This mapping prevents partner-specific error terminology from being unnecessarily exposed to CampusEats clients and provides a consistent application-level error.

5.3.9 SOAP Artifacts

The Assignment 3 deliverables are:

Artifact| Description
"integration.pdf"| Assignment report covering context, HTTP binding, discovery, and fault mapping
"partner.wsdl"| Partner service contract
"soap-request.xml"| Sample SOAP payment request
"soap-response.xml"| Sample successful SOAP response
"soap-fault.xml"| Sample SOAP fault response

The XML messages are intended to match the operation, request and response elements, binding, and endpoint described by the WSDL.

5.3.10 Assignment 3 Conclusion

Assignment 3 documents how CampusEats can communicate with an external payment partner through a SOAP-based contract. It demonstrates the role of the WSDL, SOAP Header, SOAP Body, HTTP POST binding, successful response, structured fault, and application-level fault mapping.

Important scope note: The "example.com" endpoint is illustrative. The XML artifacts describe the intended integration contract; they do not by themselves establish that a live payment gateway was contacted or that a real payment was processed.

5.4 Assignment 4 — REST API Resources and Operations

Assignment 4 focuses on REST API design and resource-oriented operations for CampusEats.

REST resources provide a way for clients to interact with application data using HTTP methods. Resources should have clear identifiers and consistent request and response structures.

The assignment materials cover the following API design concepts:

- Resource-oriented endpoint design.
- Appropriate use of HTTP methods.
- Request validation.
- Creation and management of orders.
- Consistent handling of invalid requests.
- Idempotency considerations for operations that may be retried.

For example, a REST API may use "GET" to retrieve a resource and "POST" to create a new resource. The exact endpoint paths and request schemas should be taken from the project's actual API implementation.

Outcome: The REST API work provides a structured interface for application operations and complements the external SOAP payment contract described in Assignment 3.

5.5 Assignment 5 — HTTP Protocol Features

Assignment 5 covers HTTP-level behaviour and features relevant to the CampusEats APIs.

The documented topics include:

HTTP methods and headers: HTTP methods communicate the intended operation, while headers carry request and response metadata.

ETag and caching: ETag headers can identify a representation version and support conditional requests. They can help avoid transferring an unchanged representation repeatedly.

Cross-Origin Resource Sharing (CORS): CORS controls which browser origins may access a server's resources under the browser's cross-origin rules.

Rate limiting: Rate limiting restricts the frequency of requests to help protect services from excessive traffic and abuse. Implementations may communicate limits and retry information through response headers.

These features should be documented according to the configuration and behaviour actually present in the project.

Outcome: The assignment demonstrates the importance of HTTP semantics, efficient resource access, browser access policies, and request control in web services.

5.6 Assignment 6 — Error Handling and Validation

Assignment 6 covers API error handling and validation.

A consistent error-handling approach makes the API easier for clients to understand and use. Instead of returning unclear messages, the service should provide an appropriate HTTP status code and a structured error response.

The relevant topics include:

- Appropriate HTTP status codes.
- Structured error responses.
- Request-field validation.
- Clear descriptions of invalid input.
- JSON and XML representations where supported.
- Consistent handling of failures across API operations.

The payment fault-mapping example from Assignment 3 illustrates a related principle: an external service's specific error can be translated into an application-level error that is easier for the client to handle.

Outcome: The assignment emphasizes predictable API failures, understandable validation messages, and consistent communication between services and clients.

6. Technologies and Tools

The project contains backend, frontend, database, documentation, and testing resources. The exact technology versions should be recorded from the repository configuration files.

The following tools and concepts are relevant to the available project artifacts:

- Git and GitHub for version control and source-code hosting.
- SQL and database-schema design for data modelling.
- REST and HTTP for application APIs.
- SOAP and WSDL for the external payment-partner contract.
- XML for SOAP request, response, and fault messages.
- Automated testing resources for checking application behaviour.
- OpenAPI documentation where applicable to the REST API.

Only technologies and versions confirmed by the actual project files should be listed as installed dependencies.

7. Setup and Execution

The repository is available at:

https://github.com/brajakshi1427-alt/campuseats

Use the following general steps to access the project.

1. Open the GitHub repository.
2. Download the repository ZIP file or clone the repository using Git.
3. Extract the ZIP file if downloaded.
4. Open the project directory.
5. Read the repository's "README.md" and any backend or frontend setup instructions.
6. Install the dependencies listed in the project's configuration files.
7. Configure any required local environment variables using the project's documented approach.
8. Start the application using the exact command specified in the repository documentation.
9. Run the available tests using the documented test command.

Example Git command:

git clone https://github.com/brajakshi1427-alt/campuseats.git

Execution note: The precise dependency-installation, database initialization, backend-start, and frontend-start commands should be copied from the repository's current README and verified on the machine used for the demonstration. They are not specified here because they depend on the actual project configuration.

8. Testing and Verification

Testing should cover both normal operations and failure scenarios.

The following table provides a checklist for recording the final verification evidence.

Test area| Expected behaviour| Evidence to include
Application startup| The application starts using the documented procedure| Terminal output or screenshot
REST resource operations| Requests return the expected response for valid input| API request and response
Request validation| Invalid input is rejected appropriately| Error response
SOAP request structure| The sample request follows the intended SOAP structure| "soap-request.xml"
SOAP success example| The sample response contains transaction ID and status| "soap-response.xml"
SOAP fault example| The fault contains the error code and description| "soap-fault.xml"
Error handling| Errors use the documented response structure| API response or test output
Automated tests| Tests complete with the reported results| Test command and output

The project notes previously recorded 31 passing pytest tests and successful validation of "openapi.yaml". Before final submission, rerun the tests and specification validation against the latest committed version, then include the resulting output or screenshots as evidence.

Sample SOAP XML files demonstrate message structure; they should not be presented as proof of a live external payment unless a real integration test was performed.

9. Documentation and Deliverables

The submission should include the project source code, relevant assignment artifacts, and the documentation needed to review the implementation.

The main deliverables include:

1. CampusEats GitHub repository.
2. Project "README.md".
3. Assignment 2 design and database artifacts.
4. Assignment 3 integration report, WSDL, SOAP request, successful response, and fault response.
5. Assignment 4 REST API materials.
6. Assignment 5 HTTP feature materials.
7. Assignment 6 validation and error-handling materials.
8. Relevant test output and screenshots.
9. This project submission report in PDF format.
10. Google Drive demonstration video link.

The final repository should contain the files referenced in this report, and its README should provide accurate instructions for accessing and running the project.

10. Submission Links

GitHub Repository

https://github.com/brajakshi1427-alt/campuseats

Google Drive Demonstration Video

[Paste the final Google Drive video link here.]

Before submitting the report, configure the Google Drive video's general access as Anyone with the link — Viewer. Open the link from another account or a private browser session to verify that the video is accessible.

11. Conclusion

CampusEats brings together the web-services concepts covered throughout CS543 Assignments 01–06. The project materials address system and database design, service integration, REST API operations, HTTP protocol features, and structured error handling.

Assignment 3 specifically documents the proposed integration between CampusEats and the CampusPay Payment Gateway through SOAP. Its artifacts include a WSDL contract, a SOAP request containing authentication information in the Header, a successful payment response, a structured SOAP Fault, and a mapping from the partner error "card_declined" to the application-level error "PAYMENT_DECLINED".

Together, the project repository and its supporting documentation provide a basis for reviewing the design, integration approach, API behaviour, and testing evidence for CampusEats.

12. Final Submission Checklist

- [ ] All six assignments are included and correspond to the original assignment instructions.
- [ ] Assignment 3 uses the CampusPay SOAP integration content described in this report.
- [ ] The WSDL and SOAP XML files are present in the repository.
- [ ] The Assignment 2 design and database artifacts are included.
- [ ] The REST API and HTTP/error-handling materials are included.
- [ ] The current source code is committed to GitHub.
- [ ] The repository README contains accurate setup and execution instructions.
- [ ] Automated tests and API specification validation have been rerun on the final version.
- [ ] Screenshots or output logs are included where required.
- [ ] The Google Drive demonstration link has been added and checked.
- [ ] Google Drive access is set to Anyone with the link — Viewer.
- [ ] The final report has been exported as ReadMe.pdf.

The team leader submits one final submission on behalf of the group.

Submitted by:
Brajakshi Mandloi — 20252651016
Vaishnavi Tiwari — 20252651061
Team ID: Group No. 10
Project: CampusEats
Course: CS543 — Web Services.